

Day 57: RNN Forward Pass

MD Mutasim Billah | 52-Week Data Science → ML/AI Roadmap | Week 9: Sequence Data & RNNs I
(Day 2 of 7)

0. Where Today Fits

Day 56 measured two failure modes and named what was missing: the SAME weights applied at every timestep, with information carried across the whole sequence. That is a Recurrent Neural Network. Today builds its forward pass from scratch. There is no backward pass yet (Day 58), so every check below verifies the forward computation or a capability -- not an accuracy number.

How to read each section

Every experiment below is laid out in the same order: **Specification** (exactly what was set up -- data, sizes, settings, what counts as success), **Why this test** (what question it answers), **Implementation** (the real code from the lesson script that produces the result), **Actual output** (copied verbatim from the run output file), and **What it means**. You should be able to predict each output from the first three before you read it.

Watch: [RNN From Scratch In Python](#)

Watch: [Vanishing Gradient Problem in Recurrent Neural Networks \(RNNs\) Explained!](#)

1. The RNN Cell and Classifier

SPECIFICATION	
Recurrence	$h_t = \tanh(x_t @ W_{xh} + h_{t-1} @ W_{hh} + b_h)$, starting from $h_0 = \text{zeros}$
Parameters	W_{xh} ($1 \times H$), W_{hh} ($H \times H$), b_h (H), plus an output layer W_{hy} ($H \times 2$), b_y (2); $H = \text{hidden_dim}$
Initialization	Gaussian scaled by $\sqrt{2 / \text{fan_in}}$; biases zero
Classifier	many-to-one: $\text{probs} = \text{softmax}(h_{T-1} @ W_{hy} + b_y)$ -- only the FINAL hidden state is read
Sequence length	NOT a constructor argument -- read from the input on every call
Training	none today -- weights stay at their random initialization

Why this design

One small equation, reused at every timestep, gives the RNN both properties Day 56 found missing. Shared weights mean a pattern is processed the same way wherever it occurs. The h_{t-1} term chains every state to the one before, so h_T depends -- structurally -- on every input back to x_0 , with no kernel-size cutoff.

IMPLEMENTATION

```
def rnn_step_forward(x_t, h_prev, Wxh, Whh, bh):
    return np.tanh(x_t @ Wxh + h_prev @ Whh + bh)          # the whole recurrence

def rnn_forward(X, h0, Wxh, Whh, bh):
    m, T = X.shape                                         # T comes from the INPUT
    H = np.zeros((m, T, Whh.shape[0]))
    h = h0
    for t in range(T):
        x_t = X[:, t:t + 1]                               # (batch, 1)
        h = rnn_step_forward(x_t, h, Wxh, Whh, bh)         # SAME weights every step
        H[:, t, :] = h
    return H

class VanillaRNNClassifier:
    def __init__(self, hidden_dim=8, n_classes=2, seed=42): # no T anywhere
        rng = np.random.RandomState(seed)
        self.Wxh = rng.randn(1, hidden_dim) * he_scale(1)
        self.Whh = rng.randn(hidden_dim, hidden_dim) * he_scale(hidden_dim)
        self.bh = np.zeros(hidden_dim)
        self.Why = rng.randn(hidden_dim, n_classes) * he_scale(hidden_dim)
        self.by = np.zeros(n_classes)

    def forward(self, X):
        m, T = X.shape
        h0 = np.zeros((m, self.hidden_dim))
        self.H = rnn_forward(X, h0, self.Wxh, self.Whh, self.bh)
        return softmax(self.H[:, -1, :] @ self.Why + self.by)
```

2. Correctness Check -- Loop vs. Hand-Unrolled

SPECIFICATION

Sizes	hidden_dim H = 2, one sequence of length 3
Values (seed 0)	Wxh = randn(1,2)*0.5, Whh = randn(2,2)*0.5, bh = randn(2)*0.1, X = randn(1,3)*0.5, h0 = zeros
Compared	rnn_forward's h1, h2, h3 against the same equation written out by hand, one line per timestep
Pass criterion	max absolute difference < 1e-12

Why this test

Without a backward pass there is no gradient to check. The available check is whether the general loop computes exactly what the equation says. The hand-computed version feeds each of ITS OWN results into the next line and never reads the loop's output, so the comparison is independent, not circular.

IMPLEMENTATION

```
H_loop = rnn_forward(X, h0, Wxh, Whh, bh)

h1_hand = np.tanh(X[:, 0:1] @ Wxh + h0 @ Whh + bh)
h2_hand = np.tanh(X[:, 1:2] @ Wxh + h1_hand @ Whh + bh) # uses h1_hand, NOT H_loop
h3_hand = np.tanh(X[:, 2:3] @ Wxh + h2_hand @ Whh + bh)

diffs = [np.max(np.abs(H_loop[:, 0, :] - h1_hand)),
         np.max(np.abs(H_loop[:, 1, :] - h2_hand)),
         np.max(np.abs(H_loop[:, 2, :] - h3_hand))]
```

ACTUAL OUTPUT

max abs difference at each timestep: ['0.00e+00', '0.00e+00', '0.00e+00']
overall max difference: 0.00e+00 (PASS -- loop and hand-unrolled recurrence agree exactly)

What it means

Exactly zero difference: the loop performs the identical floating-point operations in the identical order as the hand-written equations. The forward pass computes precisely the recurrence.

3. The Fixed-Length Limitation, Gone

SPECIFICATION

Network	ONE VanillaRNNClassifier(hidden_dim=8, seed=42), built once, never modified
Inputs	Day 56's task at T = 10, 20, and 30; 10 sequences each (5 per class, seed 42), spikes anywhere
Checked per length	the forward pass runs without error; output shape; every row of probabilities sums to 1
Contrast	Day 56's PlainFCNet raised a ValueError on any length other than the one it was built for

Why this test

It checks the capability directly rather than arguing it from the code: the same weights, unchanged, on three lengths. The probability-sum check confirms the output is still a valid distribution at every length.

IMPLEMENTATION

```
net = VanillaRNNClassifier(hidden_dim=8, seed=42)          # built ONCE
for T in [10, 20, 30]:
    X, y = make_order_dataset(n_per_class=5, T=T, pos_lo=0, pos_hi=T - 1, seed=42)
    probs = net.forward(X)
    print(f"T={T}<4} forward pass succeeded, output shape {probs.shape}, "
          f"probabilities sum to 1: {np.allclose(probs.sum(axis=1), 1.0)}")
```

ACTUAL OUTPUT

```
T=10  forward pass succeeded, output shape (10, 2), probabilities sum to 1: True
T=20  forward pass succeeded, output shape (10, 2), probabilities sum to 1: True
T=30  forward pass succeeded, output shape (10, 2), probabilities sum to 1: True
```

What it means

The output shape is (10, 2) every time -- 10 sequences, 2 class probabilities -- regardless of length, because the loop simply runs for as many steps as the input has.

4. Long-Range Sensitivity -- Does x_0 Reach h_T ?

SPECIFICATION

Network	VanillaRNNClassifier(hidden_dim=8, seed=7), untrained
Lengths tested	T = 5, 10, 20, 40, 80, 160
Input per length	one random sequence: $\text{randn}(1, T) * 0.5$, drawn in turn from a single rng (seed 0)
Perturbation	x_0 (the FIRST input) nudged by $+1e-3$ and $-1e-3$; everything else unchanged
Measured	sensitivity = $\ h_T(\text{nudged up}) - h_T(\text{nudged down})\ / (2 * 1e-3)$ -- a finite-difference estimate of how strongly the final state depends on the first input
Contrast	Conv1DNet's dependence on inputs more than k positions away was exactly zero, by construction

Why this test

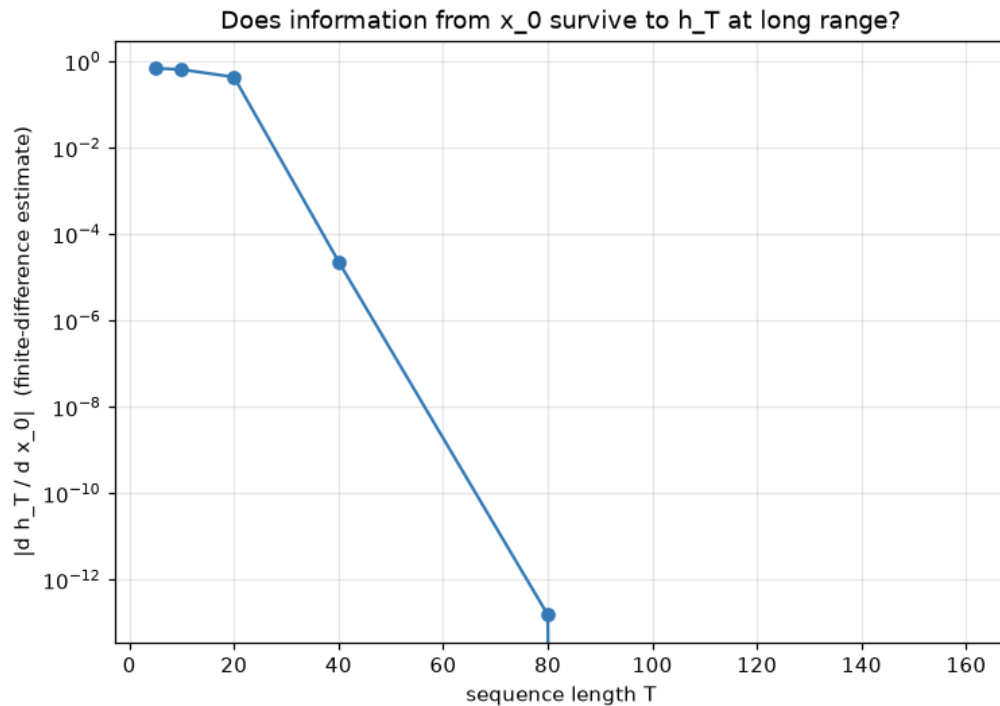
Structurally, h_T always depends on x_0 through the chain of hidden states. The open question is how STRONG that dependence is in practice. A finite difference answers it without any backward pass, and tracking it across lengths shows whether it holds, fades, or grows with distance.

IMPLEMENTATION

```
rng = np.random.RandomState(0)
net = VanillaRNNClassifier(hidden_dim=8, seed=7)
eps = 1e-3
for T in [5, 10, 20, 40, 80, 160]:
    X = rng.randn(1, T) * 0.5
    h0 = np.zeros((1, 8))
    X_plus = X.copy(); X_plus[0, 0] += eps
    X_minus = X.copy(); X_minus[0, 0] -= eps
    h_T_plus = rnn_forward(X_plus, h0, net.Wxh, net.Whh, net.bh)[: , -1, :]
    h_T_minus = rnn_forward(X_minus, h0, net.Wxh, net.Whh, net.bh)[: , -1, :]
    sensitivity = np.linalg.norm(h_T_plus - h_T_minus) / (2 * eps)
```

ACTUAL OUTPUT

T	dh_T/dx_0 (finite diff)
5	0.703337
10	0.656980
20	0.440420
40	0.000024
80	0.000000
160	0.000000



What it means

Healthy at short range (0.70 at $T=5$, 0.44 at $T=20$), then a collapse: 0.000024 at $T=40$ and EXACTLY zero by $T=160$ -- float64 can no longer represent the difference at all. ($T=160$ is missing from the log-scale plot because $\log(0)$ is undefined.) There is no hard cutoff like Conv1DNet's, but at this random initialization the influence of x_0 fades through repeated tanh and multiplication by W_{hh} .

UPDATE FROM A LATER DAY

Day 58 trained this kind of RNN and found that this measurement does NOT predict trainability on Day 56's task: an RNN trained at $T=40$ reached 0.995 test accuracy. A tiny-epsilon finite difference is a local, linear measurement. After training, the network ignored small noise (its small-signal sensitivity fell $\sim 1000\times$) while latching onto the large ± 3 spikes by saturating tanh units (their large-signal effect grew $\sim 12\times$). The numbers above are real -- but they describe an untrained network's response to tiny nudges, not what the network can learn to remember.

5. What's Still Missing

This RNN has never been trained. Learning requires a backward pass that sends gradient back through EVERY timestep, and -- because the same W_{xh} , W_{hh} , b_h are reused at each one -- sums their gradients across all T steps. That algorithm is backpropagation through time (BPTT), built and gradient-checked in Day 58.

Interview Preparation -- Technical Questions

Q: What is the single defining equation of a vanilla RNN, and how is it different from a dense layer?

A: $h_t = \tanh(x_t @ W_{xh} + h_{t-1} @ W_{hh} + b_h)$. It is applied repeatedly -- once per timestep, with the same weights each time -- and each application depends on the previous one's output.

Q: Why doesn't an RNN need a fixed sequence length?

A: None of its parameter shapes depend on T . `rnn_forward` loops for however many steps the input has -- shown directly by running one network on lengths 10, 20, and 30.

Q: Sensitivity reached exactly 0.0 at $T=160$. Does the RNN have Conv1DNet's hard cap?

A: No. The dependency exists structurally at any distance; what collapsed is its magnitude at this random initialization -- a numerical effect, not an architectural impossibility.

Q: Why verify with a hand-unrolled computation instead of a gradient check?

A: There is no backward pass to check yet. Comparing the general loop to the explicit equations serves the same purpose -- verify before trusting -- using the check actually available.

Interview Preparation -- Theory Questions

Q: What does 'parameter sharing across time' buy an RNN?

A: The same pattern is processed identically wherever it occurs in time, and the parameter count stays constant regardless of sequence length -- both things PlainFCNet lacked.

Q: Why is a vanishing FORWARD signal a warning sign for training?

A: Backpropagation through the recurrence multiplies by similar Jacobian terms, so gradients can shrink the same way. Day 58 showed the warning isn't always decisive -- a network can learn nonlinear memory that small-signal measurements miss -- but on harder tasks it is a real problem (Day 59).

Q: How can an architecture be able to represent a solution yet fail to learn it?

A: Representational capacity and trainability are different questions: gradient descent must actually find the solution, which can fail if the needed gradient signal is too weak.

Key Takeaway and What's Next

Key takeaway: a vanilla RNN is one equation reused at every timestep -- no fixed length and no hard limit on how far information travels. Verified exactly against a hand-unrolled computation and run unchanged on three lengths. At random initialization, the first input's small-signal influence on the final state collapses with distance -- a real measurement whose limits Day 58 later exposed.

What's next: Day 58 builds backpropagation through time and trains the RNN for the first time.

Suggested watch order: 1) RNN From Scratch In Python, 2) Vanishing Gradient Problem in RNNs Explained!